Can Meaning Have Universal Coordinates? Pre-Registered Negative Results, an Adapter Bottleneck, and What a Frozen Multilingual Code Can Still Do

The Sphota Authors

November 2026 (preprint draft, not yet submitted; cs.CL, cross-list cs.IR)

Abstract

We tried to build a *universal coordinate system for meaning*: a frozen, open, hierarchical code (e.g. `JYZA-7M-X0-A8`) that any text in any language receives, reachable from many embedding models through adapters, so that translations, paraphrases and different models agree on the same address. Before the decisive experiments we fixed a four-part pass/fail gate on held-out data (coverage, translation agreement, cross-model agreement, language collapse) and amended it to require at least 256 top-level cells. **No configuration passes.** The frozen release (4096 cells, multilingual-e5-small anchor) fails all four criteria; a GPU run at ~450k sentences with LaBSE and bge-m3 anchors removes language collapse and reaches 88% confident translation agreement at 256 cells, but cross-model agreement peaks at 50% (bar: 60%). A controlled adapter study then shows that much of this bottleneck is the *linear* adapter: a small residual MLP trained on the same pairs roughly doubles same-cell agreement with the anchor (five small models: 16.0% → 34.1% at 4096 cells, 32.6% → 56.1% at 256 cells; multilingual-e5-base: 30.9% → 70.5% at 256 cells), keeps cross-model retrieval at ~100% recall@10, and is still improving at 40k training pairs, while ridge is flat. An evaluation on six public domains (multilingual commands, banking support, technical Q&A, science, Indian-language reviews and code) shows what a frozen code can still do: its themes are a competitive zero-setup grouping (V-measure within 0.05–0.09 of per-dataset k-means on science, technical and review text, and 0.25–0.29 above TF-IDF there), descriptions and code share a region 77–96% of the time, and codes stay stable across runs. It also shows the limits: short informal commands share a region across languages only 36–56% of the time, far below FLORES, and narrow domains (banking intents) need per-dataset refinement. Training the space on short parallel commands lifts their agreement from 42% to 72% at 256 cells but causes language collapse elsewhere. A six-round pre-registered study of per-file refinement, each round judged once on fresh data, ends short of its bar (final round: 18 of 26 datasets within 0.05 of an oracle-k reference) yet beats the frozen themes on 38 of 42 fresh sets and groups translations together about twice as often. We measure 19 Indian languages (13 work well; Santali and Manipuri fail, Santali *confidently* wrong), report two practical lessons on labels, and release the code, artifacts, gate, negative results and every script.

1. Introduction

Embedding vectors are a good *computational* representation of meaning and a poor *data format*: a vector from model A means nothing to model B, it cannot be read or grouped in SQL, and every model upgrade forces re-embedding the corpus. A natural ambition is a shared, stable *address* for meaning, like latitude and longitude: readable, hierarchical, frozen, and the same whichever language or model produced the text.

This paper asks whether that ambition is achievable with current multilingual encoders and simple, transparent machinery, and, where it is not, what remains useful. Our answer has four parts:

- **Negative (pre-registered).** A *fine-grained* universal code (hundreds to thousands of top-level cells, stable across languages and models) is not supported, on CPU-scale or GPU-scale data, with three anchors.
- **Diagnostic.** The binding constraint at fine resolution is agreement *between models*, and a large part of it is the adapter, not the models: nonlinear adapters roughly double it.
- **Positive.** A coarse shared partition, the frozen first code block as a grouping key, and the continuous canonical space as a migration hub are useful, including on real data far from the fitting corpus.
- **Sobering.** Real, short, informal text is much harder than the parallel benchmark used to build the system; we quantify how much.

We think the negative result and the gap between benchmark and real data are worth publishing on their own. “Semantic IDs” and shared embedding spaces are active topics, and claims of a universal meaning code are easy to make on in-sample numbers; ours looked good in-sample too (81% of training sentences get a confident top-level code versus 26% held out).

The project name comes from Bhartrhari’s *sphota* (5th c. CE): meaning that bursts forth whole and stays the same across its utterances. Our results suggest it stays the same only at a coarse resolution, for now.

Contributions. 1. A complete, open method for a frozen hierarchical meaning code with calibrated per-level confidence (§3). 2. A pre-registered decision gate, an amendment closing a granularity loophole, and full results across granularity, sample design, anchor and data scale (§4–5). 3. A controlled adapter study identifying the linear adapter as a major part of the cross-model bottleneck, and nonlinear adapters that double agreement (§6). 4. A six-domain evaluation on license-clean public data, including code and Indian languages (§7, §9). 5. A six-round pre-registered study of per-file refinement on fresh data each round, reported in full although it does not meet its bar (§8), and an experiment showing what training the space on short text buys and costs (§5.4). 6. Practical lessons on labels and on releasing open ML artifacts safely (§11), and a fully reproducible release with the research protocol (Appendices A and C).

2. Related work

Semantic IDs. TIGER (Rajput et al., 2023) quantizes item embeddings with an RQ-VAE into codeword tuples for generative retrieval; DSI (Tay et al., 2022) uses hierarchical k-means document IDs; Singh et al. (2024) study Semantic IDs for ranking. These codes are private to one system and refit per catalogue, so they never face the cross-language and cross-model stability test that a *universal* code must pass. Codebook collapse in RQ-VAE is mitigated by EMA updates and dead-code revival (Zeghidour et al., 2021; Lee et al., 2022).

Embedding translation. vec2vec (Jha et al., 2025) translates between embedding spaces without paired data, in support of the Platonic representation hypothesis (Huh et al., 2024). Supervised linear maps between embedding spaces are a long-standing baseline (e.g. cross-lingual word embedding mapping). Our results are compatible with the hypothesis at the level of *continuous* geometry (migration works) while showing that *discrete* agreement is far weaker, and that the gap between linear and nonlinear maps matters at fine resolution.

Language-agnostic spaces. LASER, LaBSE (Feng et al., 2022) and SONAR (Duquenne et al., 2023) embed many languages jointly; Large Concept Models (2024) operate in SONAR space. We quantify how much of their cross-lingual alignment survives discretization.

Topic models and clustering benchmarks. BERTopic (Grootendorst, 2022) clusters embeddings and labels clusters with c-TF-IDF, refitting per corpus. MTEB (Muennighoff et al., 2023) evaluates embedding clustering with V-measure on per-dataset k-means. Our positive result is close to a *frozen, multilingual, shared* BERTopic-style partition; we report it against MTEB-style per-dataset clustering so the cost of freezing is explicit.

3. Method

3.1 Anchor encoder. `intfloat/multilingual-e5-small` (MIT, 384-d, ~100 languages), input `"query: " + text`, mean pooling, L2 normalization. Chosen in a bake-off against bge-m3, LaBSE and paraphrase-multilingual-MiniLM for speed, size and an exact ONNX export (max abs diff $7{\cdot}10^{-8}$ versus PyTorch); it runs on a laptop CPU and in the browser. §5 revisits the choice.

3.2 Canonical space (meaning-LDA). With unit vectors `x`, `y = (x − μ) W`: PCA to 384 directions, then the generalized eigenvectors of `(S_total, S_within + reg)`, where `S_within` is the scatter of translations of the same sentence (FLORES-200 dev, 800 meanings × 12 languages; 7.2k Tatoeba pairs), shrunk toward isotropy (`reg = 0.05`). The top 256 directions are scaled so within-meaning noise has unit variance. PCA alone gives 34.1% cross-lingual L1 agreement versus 46.1% with meaning-LDA (4096 cells).

3.3 Quantizer and code. Residual k-means with 4 levels (4096·256·256·256) on 77.6k sentences (Wikipedia sample in 12 languages + FLORES dev). Codes are Crockford base32 (widths 4/2/2/2) after a published affine

permutation, so 85% of single-symbol typos are rejected without a checksum. A separate layer of 32 coarse *regions* (k-means on the canonical space) is stored alongside, outside the frozen artifact.

3.4 Confidence. At each level, a softmax over the 16 nearest centroids' squared distances, with a temperature fitted by maximum likelihood to predict the *held-out translation's* symbol; the product over levels is the prefix confidence.

3.5 Adapters. Other models enter the canonical space through maps fitted on 40k paired texts (registry: all-MiniLM-L6-v2, bge-small-en-v1.5, gte-small, paraphrase-multilingual-MiniLM-L12-v2, LaBSE). The frozen release uses ridge regression on L2-normalized model vectors. Set *v1.1* adds a residual network on the input, $y = xA + b + \mathrm{gelu}(xW_1 + c_1)W_2 + c_2$ (one 1024-unit hidden layer, tanh-approximated GELU, MSE loss, AdamW, 40 epochs), shipped as a NumPy-only file (float16 hidden weights) and certified with the NumPy forward pass. An unsupervised vec2vec-style adapter failed at our CPU budget (top-1 retrieval 0% versus 96.5% for ridge on the same texts).

4. The decision gate

Owner review of the first release concluded that in-sample numbers did not support a "GPS for meaning" claim. We therefore fixed the following gate **before** any v2 run. All criteria use held-out FLORES-200 devtest sentences in 12 languages, never used to fit the space, codebooks, calibration, labels or adapters.

#	Criterion	Definition	Bar
1	Coverage	share of held-out sentences (all 12 languages) with L1 confidence ≥ 0.7	≥ 60%
2	Translation agreement	among en ↔ x pairs whose English code has L1 confidence ≥ 0.7, share with the same L1 code (mean over 11 languages)	≥ 80%
3	Cross-model agreement	same English text, certified adapter vs anchor, same L1 code (mean over adapters)	≥ 60%
4	Language collapse	per language, share of its sentences in L1 cells whose training members are ≥ 80% that language (English excluded)	≤ 5% for every language

Amendment. A tiny-scale dry run "passed" all four criteria with 16 top-level cells. A gate that can be passed by making the partition coarse says nothing about coordinates, so the gate must be met with **≥ 256 top-level cells**, on all 12 languages, with the cell count declared before the run.

5. Gate results

5.1 CPU-scale sweep. Along a re-clustering sweep of the frozen space, every build at 256, 1024 or 4096 cells fails all four criteria; the closest configurations (3 of 4) use 32–64 cells:

Build	L1 cells	Coverage	Translation, confident	Cross-model	Worst collapse	Pass
v1 frozen release	4096	29.4%	65.1%	15.7%	33.8% (pt)	0/4
v1 re-clustered	1024	42.6%	69.7%	23.1%	44.2% (pt)	0/4
v1 re-clustered	256	49.2%	74.4%	32.8%	54.6% (pt)	0/4
v1 re-clustered	64	65.0%	78.3%	57.1%	0.0%	2/4
v1 re-clustered	32	69.0%	79.3%	64.5%	0.0%	3/4

Build	L1 cells	Coverage	Translation, confident	Cross-model	Worst collapse	Pass
LaBSE anchor (20k corpus)	32	66.5%	88.7%	53.0%*	0.0%	3/4

* reduced cross-model panel. Stability is bought with coarseness: each 4× refinement costs roughly 10–25 points, cross-model agreement degrading fastest.

5.2 Language collapse. At ≥ 256 cells, k-means spends cells on language identity: 52.8% of Portuguese test sentences fell into Portuguese-dominated cells (≤ 0.2% for other languages). The Portuguese Wikipedia sample is unusually homogeneous; a diversified sample cut the worst collapse at 4096 cells only from 33.8% to 26.8%, because the e5 anchor keeps a separable Portuguese direction. Collapse is invisible in average agreement and should be reported separately.

5.3 GPU run (medium scale). One bounded run (Kaggle T4, 12 languages, ~450k sentences, diversified per-language sample), against the pre-registered bar:

Run	Anchor	Adapters	L1 cells	Coverage (≥ 60%)	Translation, confident (≥ 80%)	Cross-model (≥ 60%)	Worst collapse (≤ 5%)	Pass
1	multilingual-e5-small	linear	256	46.3%	74.4%	31.3%	40.9%	0/4
1	multilingual-e5-small	linear	1024	39.3%	71.6%	24.7%	31.6%	0/4
1	multilingual-e5-small	linear	4096	26.7%	66.0%	16.2%	34.2%	0/4
1	LaBSE	linear	256	69.4%	83.5%	40.1%	0.2%	3/4
1	LaBSE	linear	1024	55.2%	83.9%	29.3%	0.1%	2/4
1	LaBSE	linear	4096	46.9%	78.4%	17.7%	0.4%	1/4
1	bge-m3	linear	256	67.2%	88.0%	50.2%	0.1%	3/4
1	bge-m3	linear	1024	62.8%	82.6%	38.0%	0.1%	3/4
1	bge-m3	linear	4096	54.5%	78.6%	25.3%	0.2%	1/4
2	LaBSE (+ MASSIVE scatter)	MLP, 20k–40k pairs	256	56.0%	82.0%	45.6%	0.2%	2/4
2	LaBSE (+ MASSIVE scatter)	MLP, 20k–40k pairs	1024	47.0%	77.6%	35.0%	0.3%	1/4
2	LaBSE (+ MASSIVE scatter)	MLP, 20k–40k pairs	4096	27.4%	80.1%	26.2%	0.1%	2/4
2	bge-m3 (+ MASSIVE scatter)	MLP, 40k pairs	256	61.4%	86.8%	59.2%	0.0%	3/4
2	bge-m3 (+ MASSIVE scatter)	MLP, 40k pairs	1024	52.9%	82.2%	46.2%	0.2%	2/4
2	bge-m3 (+ MASSIVE scatter)	MLP, 40k pairs	4096	38.0%	80.6%	35.8%	0.3%	2/4

Run 1 (linear adapters): stronger anchors remove language collapse and lift confident translation agreement to 88.0% (bge-m3, 256 cells), but cross-model agreement peaks at 50.2%. Run 2, registered after the adapter study of §6, changed two things at once: nonlinear adapters and short parallel commands in the scatter (§5.4). bge-m3 at

256 cells then passes coverage (61.4%), translation (86.8%) and collapse (0.0%) and misses cross-model by 0.8 points (59.2%; per panel model 55.8– 62.0%). We had predicted, from the CPU experiment of §5.4, that the extra scatter would cause language collapse; it did not with these anchors, so that prediction was wrong. **The gate fails in both runs.** Run 3 (one change: adapters trained on 120k instead of 40k pairs, motivated by the scaling curve in §6) was registered before it ran (Appendix C).

5.4 Can the space learn short text? §7 shows that short informal commands agree across languages far less often than FLORES sentences. The canonical space decides what counts as "the same meaning" through the translation scatter of meaning-LDA (§3.2), which so far held only FLORES and Tatoeba. We added parallel MASSIVE *train* commands (aligned across the 12 languages; texts that occur in the test split removed) and measured held-out MASSIVE *test* commands:

MASSIVE meanings in the scatter	Cells	Short commands, same cell as English	Coverage	Translation, confident	Cross-model	Worst collapse
0 (baseline)	32	51.9%	63.9%	78.8%	62.3%	0.1% (ar)
0 (baseline)	256	42.4%	48.1%	75.8%	32.9%	0.0% (hi)
0 (baseline)	1024	40.6%	37.4%	71.7%	21.8%	51.0% (pt)
800	32	69.5%	60.9%	80.8%	64.5%	0.1% (ar)
800	256	53.5%	43.2%	75.8%	35.7%	55.3% (pt)
800	1024	49.3%	31.1%	69.8%	21.5%	61.5% (pt)
2,500	32	62.1%	55.7%	80.2%	62.0%	0.0% (hi)
2,500	256	55.0%	37.6%	72.1%	35.1%	75.2% (pt)
2,500	1024	58.4%	30.8%	64.7%	24.0%	68.7% (pt)
11,492	32	83.8%	58.0%	77.9%	66.5%	0.0% (hi)
11,492	256	72.3%	30.7%	70.8%	38.6%	84.9% (pt)
11,492	1024	50.3%	19.1%	69.2%	28.2%	77.7% (pt)

The space learns the new text type quickly: at 256 cells, short commands land in the same cell as their English version 72% of the time instead of 42%. But at ≥ 256 cells every amount we tried induces language collapse elsewhere (Portuguese Wikipedia, the homogeneous sample of §5.2), and FLORES coverage falls. At 32 cells, 800 extra meanings is a clean gain (all four gate criteria pass, short-command agreement 52% → 69%), which is evidence for a better coarse layer but not a valid gate result. The scatter is a lever, not a free lunch: it moves agreement toward the text types it contains.

6. The adapter bottleneck

If cross-model agreement is the binding constraint, how much of it is the adapter? We held the frozen space, the training texts and the held-out FLORES test fixed and varied only the adapter (agreement = the other model's text, mapped through the adapter, lands in the anchor's cell for the same text; all 12 languages for multilingual models, English otherwise):

Model	Adapter	Same cell (4096)	Same cell (256)	Same region (32)
all-MiniLM-L6-v2	rotation	6.1%	7.7%	55.7%
all-MiniLM-L6-v2	linear (v1)	16.6%	33.6%	64.8%
all-MiniLM-L6-v2	residual MLP	35.8%	57.5%	70.5%
all-MiniLM-L6-v2	MLP + L1 code loss	33.9%	50.9%	58.5%
all-MiniLM-L6-v2	MLP + L1/region loss	35.2%	51.0%	64.4%
bge-small-en-v1.5	rotation	10.5%	15.0%	61.0%

Model	Adapter	Same cell (4096)	Same cell (256)	Same region (32)
bge-small-en-v1.5	linear (v1)	13.8%	31.1%	66.5%
bge-small-en-v1.5	residual MLP	36.6%	62.1%	76.0%
bge-small-en-v1.5	MLP + L1 code loss	36.1%	53.9%	64.6%
bge-small-en-v1.5	MLP + L1/region loss	37.1%	54.5%	69.9%
gte-small	rotation	10.7%	14.9%	62.8%
gte-small	linear (v1)	5.7%	16.7%	64.2%
gte-small	residual MLP	33.3%	57.5%	71.2%
gte-small	MLP + L1 code loss	32.6%	48.0%	58.1%
gte-small	MLP + L1/region loss	34.0%	48.7%	65.4%
paraphrase-multilingual-MiniLM-L12-v2	rotation	22.7%	38.3%	60.2%
paraphrase-multilingual-MiniLM-L12-v2	linear (v1)	24.5%	42.5%	67.3%
paraphrase-multilingual-MiniLM-L12-v2	residual MLP	32.8%	50.4%	69.8%
paraphrase-multilingual-MiniLM-L12-v2	MLP + L1 code loss	33.1%	46.7%	62.9%
paraphrase-multilingual-MiniLM-L12-v2	MLP + L1/region loss	33.0%	46.6%	69.0%
LaBSE	rotation	12.5%	25.5%	58.6%
LaBSE	linear (v1)	19.4%	39.2%	65.3%
LaBSE	residual MLP	31.9%	52.9%	71.1%
LaBSE	MLP + L1 code loss	35.6%	53.3%	67.7%
LaBSE	MLP + L1/region loss	35.1%	52.9%	71.4%
multilingual-e5-base	rotation	19.9%	37.2%	68.1%
multilingual-e5-base	linear (v1)	14.3%	30.9%	69.0%
multilingual-e5-base	residual MLP	51.0%	70.5%	82.5%
multilingual-e5-base	MLP + L1 code loss	50.7%	65.6%	77.6%
multilingual-e5-base	MLP + L1/region loss	50.1%	65.1%	82.2%
bge-base-en-v1.5	rotation	11.9%	13.7%	60.6%
bge-base-en-v1.5	linear (v1)	16.2%	35.0%	67.9%
bge-base-en-v1.5	residual MLP	41.1%	64.3%	75.7%
bge-base-en-v1.5	MLP + L1 code loss	39.7%	54.8%	66.5%
bge-base-en-v1.5	MLP + L1/region loss	39.1%	55.2%	69.0%
distiluse-base-multilingual-cased-v2	rotation	20.7%	33.2%	62.3%
distiluse-base-multilingual-cased-v2	linear (v1)	28.9%	46.0%	68.3%
distiluse-base-multilingual-cased-v2	residual MLP	36.9%	52.3%	69.1%
distiluse-base-multilingual-cased-v2	MLP + L1 code loss	37.2%	49.9%	65.0%
distiluse-base-multilingual-cased-v2	MLP + L1/region loss	36.8%	49.6%	68.1%
mean, 5 small models	**linear (v1)**	**16.0%**	**32.6%**	**65.6%**

Model	Adapter	Same cell (4096)	Same cell (256)	Same region (32)
mean, 5 small models	**residual MLP**	**34.1%**	**56.1%**	**71.7%**
mean, all 8 models	**linear (v1)**	**17.4%**	**34.4%**	**66.7%**
mean, all 8 models	**residual MLP**	**37.4%**	**58.4%**	**73.2%**

The gain holds, and grows, for three larger models added later (D-045): multilingual-e5-base goes from 14.3% to 51.0% at 4096 cells and from 30.9% to 70.5% at 256 cells; bge-base-en-v1.5 from 35.0% to 64.3% at 256 cells; distiluse-multilingual-v2, already better aligned linearly, gains less (46.0% → 52.3%). For two of the three, cross-model agreement at 256 cells clears the gate's 60% bar in this setting (re-clustered cells, FLORES devtest), which the full gate has not yet been re-run to confirm.

A residual MLP roughly doubles fine-level agreement and brings the gate's 256-cell level from 32.6% to 56.1%, within four points of the 60% bar, *with the small e5 anchor on CPU-scale data*. Training the adapter directly on the anchor's codes (cross-entropy over the codebook at L1, or at L1 and the region level) did not beat plain MSE on average: it gains a little at L1 for multilingual models but gives up coarse agreement. Rotation-only (Procrustes) is far worse than ridge. Data scaling (same cell, 4096 cells, all languages):

Training pairs	all-MiniLM-L6-v2: linear	MLP	paraphrase-multilingual-MiniLM-L12-v2: linear	MLP
2,500	4.7%	7.0%	23.9%	27.0%
5,000	5.0%	8.2%	23.9%	27.3%
10,000	5.1%	9.9%	24.3%	29.4%
20,000	5.3%	10.9%	24.6%	31.7%
40,000	5.2%	11.9%	24.5%	32.8%

Ridge is flat from 2.5k to 40k pairs; the MLP is still improving at 40k. The shipped v1.1 adapters, certified with the NumPy forward pass, preserve migration recall:

Model	Same cell, English (v1 → v1.1)	Same cell, 12 languages (v1 → v1.1)	Migration recall@10 (v1 → v1.1)	Grade
all-MiniLM-L6-v2	16.6% → 35.8%	5.2% → 8.5%	99.9% → 99.9%	certified
bge-small-en-v1.5	13.8% → 36.6%	4.9% → 9.8%	100.0% → 100.0%	certified
gte-small	5.7% → 33.3%	3.4% → 9.6%	99.8% → 100.0%	certified
paraphrase-multilingual-MiniLM-L12-v2	23.3% → 35.4%	24.5% → 32.8%	100.0% → 100.0%	certified-multilingual
LaBSE	19.2% → 32.8%	19.4% → 31.9%	100.0% → 100.0%	certified-multilingual
multilingual-e5-base	n/a → 52.8%	n/a → 51.0%	n/a → 100.0%	certified-multilingual
bge-base-en-v1.5	n/a → 41.1%	n/a → 12.4%	n/a → 100.0%	certified

We did not re-run the full gate with nonlinear adapters and a strong anchor at GPU scale; given §5.3 and this section, that is the most promising open experiment.

7. Beyond FLORES: six domains

FLORES sentences are formal, parallel and close to the Wikipedia fitting corpus. We therefore evaluated on public, license-clean datasets far from it (Appendix B): multilingual voice-assistant commands (MASSIVE, 59 intents, 14 languages, parallel), banking support queries (Banking77, 77 intents), Stack Exchange titles (technical Q&A, site labels), arXiv titles (science, 12 top-level categories), product reviews in 13 Indian languages (IndicReviews, 15 categories, parallel) and code (CodeSearchNet, six programming languages).

7.1 Grouping. V-measure (the MTEB clustering metric) and purity of Sphota's fixed groupings (themes = first code block; regions = 32), next to k-means fitted on each dataset's own anchor embeddings at the same number of

groups (a reference that *sees the data*), at 32 groups, at the true number of labels, and TF-IDF + k-means:

Domain	Lang.	n	Labels	Sphota themes: V / purity (groups)	k-means, same #groups: V / purity	Sphota regions: V	k-means 32: V	k-means true k: V	TF-IDF: V
Conversational (MASSIVE)	en	2,974	59	0.530 / 51% (485)	0.650 / 77%	0.291	0.618	0.660	0.493
Conversational (MASSIVE)	hi	2,974	59	0.469 / 43% (407)	0.617 / 70%	0.237	0.515	0.583	0.363
Conversational (MASSIVE)	ta	2,974	59	0.401 / 35% (312)	0.576 / 62%	0.194	0.417	0.468	0.367
Customer support (Banking77)	en	3,080	77	0.428 / 23% (166)	0.738 / 67%	0.161	0.593	0.706	0.591
Technical Q&A (Stack Exchange)	en	5,000	20	0.422 / 54% (578)	0.513 / 78%	0.210	0.607	0.570	0.132
Science (arXiv titles)	en	5,000	12	0.210 / 46% (235)	0.272 / 64%	0.110	0.309	0.302	0.063
Product reviews (IndicReviews)	hi	1,000	15	0.404 / 63% (196)	0.478 / 78%	0.246	0.463	0.448	0.159
Product reviews (IndicReviews)	ta	1,000	15	0.378 / 60% (204)	0.441 / 72%	0.247	0.382	0.368	0.168
Product reviews (IndicReviews)	bn	1,000	15	0.412 / 64% (211)	0.465 / 76%	0.266	0.433	0.414	0.152

Sphota's frozen themes are a competitive zero-setup grouping: close to per-dataset k-means on science, reviews and technical Q&A, and far above keyword clustering there. The cost of freezing is largest on short commands and on banking, a narrow domain whose 77 intents fall into relatively few general-purpose cells. Regions are too coarse for these label sets. In exchange, theme codes are identical across runs and files, which per-dataset clustering cannot offer; a per-file refinement step on top of the stable codes is the natural product answer.

7.2 Cross-lingual agreement on real text. Share of parallel items whose translation lands in the same theme or region as the pivot language:

Data (pivot)	Language	Same theme	Same region
MASSIVE (English)	hi	36.0%	47.2%
MASSIVE (English)	ta	25.1%	38.5%
MASSIVE (English)	te	25.2%	35.9%
MASSIVE (English)	kn	27.3%	38.4%
MASSIVE (English)	ml	31.5%	43.4%
MASSIVE (English)	bn	28.3%	37.8%
MASSIVE (English)	ur	29.4%	42.4%
MASSIVE (English)	es	45.6%	56.1%
MASSIVE (English)	fr	44.9%	54.1%
MASSIVE (English)	de	41.5%	52.5%
MASSIVE (English)	zh-CN	31.9%	50.1%
MASSIVE (English)	ar	30.7%	44.4%

Data (pivot)	Language	Same theme	Same region
MASSIVE (English)	ru	42.0%	51.8%
IndicReviews (Hindi)	bn	46.0%	70.5%
IndicReviews (Hindi)	mr	45.9%	71.8%
IndicReviews (Hindi)	te	44.1%	69.4%
IndicReviews (Hindi)	ta	38.9%	68.7%
IndicReviews (Hindi)	ur	46.0%	72.0%
IndicReviews (Hindi)	gu	46.1%	69.6%
IndicReviews (Hindi)	kn	44.5%	70.8%
IndicReviews (Hindi)	or	42.2%	68.6%
IndicReviews (Hindi)	ml	45.9%	72.8%
IndicReviews (Hindi)	pa	45.4%	70.3%
IndicReviews (Hindi)	as	35.8%	63.1%
IndicReviews (Hindi)	bd	9.6%	47.5%

Real, short, informal commands are much harder than FLORES: the same command shares the English region 36–56% of the time (FLORES: ~72%). Longer review sentences hold up better (63–73% against Hindi). Bodo, which the anchor does not cover, fails.

7.3 Cross-model agreement on real text. Multilingual MiniLM through its adapters versus the anchor, on MASSIVE:

MASSIVE language	Adapter	Same theme as anchor	Same region as anchor
en	v1	17.2%	51.1%
en	v1.1	31.6%	56.4%
hi	v1	15.1%	43.7%
hi	v1.1	28.3%	51.9%

The nonlinear adapters' gain carries over to real data (about twice the same-theme agreement).

7.4 Code. A docstring and its function (docstrings stripped from the code in this benchmark):

Language	n	Doc and code: same theme	same region	Retrieval R@1 / R@10, raw anchor	canonical space
python	1000	54.5%	87.3%	71.0% / 93.8%	68.8% / 93.0%
java	1000	50.2%	95.7%	56.5% / 89.6%	53.0% / 84.9%
javascript	1000	35.8%	76.7%	51.5% / 75.2%	50.5% / 76.1%
go	1000	50.5%	89.2%	68.9% / 92.8%	67.8% / 92.4%
php	1000	33.5%	84.9%	61.1% / 88.6%	58.5% / 85.4%
ruby	1000	40.3%	83.0%	59.7% / 85.9%	57.6% / 83.7%

A text-trained anchor places code and its description in the same coarse region 77–96% of the time, and the canonical projection costs only 0–5 points of docstring→code recall@10 against raw anchor vectors (the candidate pool is the 1000 sampled pairs per language).

8. Per-file refinement: six pre-registered rounds

Frozen themes are stable but generic; one file is often narrow (all banking support) or mixed (the same complaint in four languages). We therefore studied *per-file refinement*: groups fitted to one file, each tied to a stable theme code, with every message's own code unchanged. Before building anything we fixed four targets (D-033): **R1** grouping quality within 0.05 V-measure of k-means that is *told the true number of labels* (or uses our number of

groups, whichever is better), on every dataset; **R2** determinism (row order cannot change the groups; ARI ≥ 0.95); **R3** in files mixing two languages, translations share a refined group at least as often as they share a fixed theme; **R4** ≤ 2 s for 3,000 messages.

Protocol. Each dataset was split once into dev (30%) and test (70%). Candidates were chosen on dev only, and each round evaluated **only the dev winner, once, on data no method had touched**: round 1 on the test splits, rounds 2–6 on new confirmation sets (more MASSIVE languages, the MASSIVE validation split, new Stack Exchange and arXiv splits, MasakhaNEWS train, FLORES devtest). Every amendment was committed before the run it affected, and a pipeline smoke test that accidentally reported test numbers was disclosed (D-034). 95% bootstrap intervals accompany each headline number in the result files.

Round	Winner (chosen on dev)	Fresh test data	R1 quality	R2 stable	R3 mixed languages	R4 speed
1	M3_auto+centre	test splits (22 sets)	15/22	fail	5/8	pass
2	M4_kmeans+centre	confirmation set 1	16/25	pass	7/7	pass
3	M4_kmeans_2sqrt+centre	set 2 (13 new MASSIVE languages)	12/16	pass	1/4	pass
4	M4_kmeans_2sqrt[canonical,script]	set 3 (24 more languages)	26/27	pass	2/6	pass
5	M4_kmeans_2sqrt[raw,language_masked]	set 4 (MASSIVE validation)	15/16	pass	9/9	pass
6	M4_kmeans_2sqrt[raw,language_masked]	set 5 (Stack Exchange, arXiv, MasakhaNEWS train, FLORES)	18/26	pass	11/12	pass

Each round fixed the failure the previous one exposed: an ordering bug (R2), a silhouette rule that picked too few groups (R1), script-based language correction that cannot tell English from Italian (R3; replaced by a 51-language identifier trained on MASSIVE *train*, 94% accurate), and a dev set that lacked same-script pairs and so could not see that failure (a methodological lesson: the dev set must contain the failure mode being fixed). **The final method does not meet the bar** (round 6: R1 18/26, R3 11/12). The residual R1 failure is structural: the reference may use the true number of labels, and no label-free rule fits both fine label sets (59–77 intents) and coarse, well-separated ones (12–17 Stack Exchange sites, 7 news topics).

Against what users had before (the fixed themes), the refined groups win on 38 of 42 fresh sets (mean V-measure gain +0.05 to +0.10; Banking77 0.42 → 0.69–0.74), keep translations together about twice as often, run in 0.1 s, and lose only on four languages the anchor barely knows (−0.01 to −0.02). We ship them as the default for files of ≥ 200 messages, in the library and in the browser (a TypeScript port that agrees exactly with the Python implementation on the same vectors).

9. Indian languages

FLORES-200 devtest for 19 Indian languages against English (frozen s1 space):

Language	Region = English	Agreement when both confident	First block = English	Retrieval@1	Top-region share
English (reference)					36.1%
Hindi	76.4%	92.2%	52.7%	99.9%	39.8%
Bengali	71.9%	89.1%	47.3%	100.0%	41.0%

Language	Region = English	Agreement when both confident	First block = English	Retrieval@1	Top-region share
Marathi	72.4%	89.5%	48.2%	99.5%	39.7%
Telugu	71.3%	90.1%	44.7%	99.6%	37.5%
Tamil	73.1%	90.4%	45.0%	99.3%	39.3%
Urdu	72.0%	88.3%	45.1%	99.5%	42.1%
Gujarati	71.8%	87.8%	46.5%	99.6%	39.7%
Kannada	73.7%	90.7%	50.3%	99.2%	39.9%
Odia	72.0%	86.8%	45.1%	99.4%	40.8%
Malayalam	72.4%	91.1%	47.8%	99.7%	37.6%
Punjabi	70.2%	87.5%	43.3%	99.1%	38.6%
Assamese	67.4%	82.7%	37.0%	97.9%	45.1%
Maithili	71.0%	89.2%	46.2%	99.8%	43.5%
Nepali	71.2%	89.7%	47.0%	99.2%	40.8%
Sindhi	62.5%	78.7%	34.8%	98.3%	47.8%
Sanskrit	63.2%	82.2%	34.1%	95.7%	52.3%
Kashmiri (Arabic)	61.1%	77.9%	30.4%	91.7%	49.3%
Manipuri (Bengali script)	45.4%	55.6%	15.8%	62.0%	68.6%
Santali (Ol Chiki)	36.1%	34.9%	1.0%	2.6%	97.5%

Thirteen languages work well (87–92% region agreement when both codes are confident, retrieval@1 ≥ 99%), four partly. Manipuri (Bengali script) and Santali (Ol Chiki) fail. Santali is *confidently wrong*: 94% of its sentences get region confidence ≥ 0.7 while 97.5% land in a single region, so calibrated confidence does not protect users on scripts the anchor never saw. The library therefore warns on Ol Chiki and Meetei Mayek input.

10. Why fine-grained universal codes fail

Three effects compound. (1) **Boundary mass:** equivalent texts are small clouds, not points; the fraction of clouds cut by some cell boundary rises with the number of cells. (2) **Adapter error:** each adapter adds its own displacement, which at fine granularity exceeds the cell radius. §6 shows this term is partly reducible: nonlinear adapters halve the gap at 256 cells. (3) **Residual language signal:** at fine granularity k-means can profitably spend cells on language. Confidence and alternates make (1) visible but do not remove it. A code can be calibrated and honest about its uncertainty and still not be universal.

11. What works, and practical lessons

Coarse regions and themes. A frozen first code block is a stable grouping key for analytics (SQL `GROUP BY`, dashboards, trends over time), routing and cache keys, with finer levels as optional detail carrying explicit confidence.

Migration through stored canonical vectors. Documents indexed with model X through its adapter and queried with the anchor reach recall@10 of 96.5–99.1% (ridge) and ~100% (v1.1) on FLORES, versus 100% for re-embedding; codes alone give 10–28%. Storing the 256-d canonical vector lets an organisation change models without re-embedding, with N adapters instead of N^2 pairwise maps.

Lesson 1: labels from a general corpus mislead out of domain. Cell names generated from Wikipedia members describe everyday text badly (“my order hasn’t arrived” → *Kali Yuga Tenets and Authors*; “Messi scored twice in the final” → *Golf Championships*, at L1 confidence 0.79). Hiding labels below a confidence threshold left 7 of 48 support messages labelled, and those 7 were still wrong. Naming each group by its most confident member *from the user’s own data* works better.

Lesson 2: machine-translated labels need a semantic check. Script and length filters miss fluent nonsense (Bengali *Automotive Innovations* → "Unique Violence"). A ranking test (the translated name or its back-translation must point to its own label among all labels) removed 30 of ~550 names across 11 languages.

Lesson 3: releasing open ML artifacts safely. Pin every downloaded model to a hub commit and verify file hashes (a silently updated model changes "frozen" codes); never load pickles or remote model code; serve browser runtimes from the site itself (our in-browser encoder's library fetched a development build of its runtime from a CDN until a Content Security Policy caught it); build wheels in an isolated job with hash-pinned tools and refuse any wheel that is not byte-identical to the tracked source (the 2026 compromises of a widely used Python package shipped a `.pth` startup hook); publish with provenance attestations and no stored tokens.

12. Limitations

Most experiments are CPU-scale (≤ 80k fitting sentences, 12 languages); one GPU run reached ~450k sentences. We did not combine nonlinear adapters with a strong anchor at GPU scale. Region and cell labels are machine-generated and not natively reviewed. The six domains are English-heavy outside MASSIVE and IndicReviews; IndicReviews is machine- or human-translated parallel text whose translation process we did not audit. Per-dataset k-means is a reference, not a ceiling; stronger clustering (e.g. HDBSCAN) was not tried. FLORES dev was used to fit the space, so no FLORES-derived dataset (e.g. SIB-200) appears in §7. The refinement study spent six evaluation rounds on related benchmark families; although every round used items no method had seen, the method was adapted between rounds, so later rounds are less independent than the first. The language identifier is trained on MASSIVE train, a sibling of the MASSIVE test and validation sets used for evaluation.

13. Conclusion

Can meaning have universal coordinates? At fine resolution, with today's small multilingual encoders, our pre-registered answer is **no**: no build passes a fixed, held-out gate at ≥ 256 cells. But the failure is not a wall: a large part of it is the linear adapter, and nonlinear adapters close half the gap at the gate's granularity with no new data. At coarse resolution the answer is a qualified **yes**, with an important caveat measured here for the first time on real text: short informal messages agree across languages far less often than parallel benchmarks suggest. Two levers we measured move these numbers, each with a price: teaching the space short parallel text (a 30-point gain for short commands, paid for by language collapse elsewhere at fine granularity), and fitting groups per file (better than frozen themes on 38 of 42 fresh sets, but short of an oracle that knows how many categories the user has in mind). We recommend that work on semantic IDs and shared meaning spaces report held-out, cross-language and cross-model agreement *as a function of granularity*, together with language collapse, evaluate on real text outside the fitting distribution, and fix the bar before running.

Appendix A. Reproducibility

Every table is generated from `bench/results/*.json` by `scripts/paper_tables.py`; the PDF and LaTeX are built by `scripts/build_paper.py`. Commands (all accept `--scale {tiny,small,full}`):

Section	Command	Result file
§5.1 sweep	`python -m sphota.build.gate sweep --scale small`	`l1-sweep.json`, `anchor-check.json`
§5.3 GPU gate	`notebooks/kaggle_gpu_gate.ipynb` or `bash scripts/gpu_build_v2.sh --scale medium`	`v2-gate-medium-*.json`
§6 adapters	`python -m sphota.build.adapter_study --scale small`	`adapter-study-small.json`
§6 v1.1	`python -m sphota.build.adapters_v11 --`	`adapters-v1.1-small.json`

Section	Command	Result file
	`scale small`	
§7 domains	`python -m sphota.build.domains --scale small`	`domains-small.json`
§9 Indian languages	`python -m sphota.build.indic --scale small`	`indic-small.json`
§5.4 short text	`python -m sphota.build.gate v2mini --scale small --sizes 32 256 1024 [--extra-parallel massive --extra-n N]`	`shorttext-*.json`
§8 refinement	`python -m sphota.build.refine_study --scale small [--round 2..6]`	`refine-study-small.json`, `refine-round*-small.json`

Code and artifacts: Apache-2.0. Decisions and their rationale: `DECISIONS.md` (D-015 – D-032).

Appendix C. Research protocol and timeline

All bars, splits, candidate sets and selection rules were committed to `DECISIONS.md` before the runs they govern; the git history timestamps each entry before its result file. Decision gate: D-017/D-018 (fixed before any v2 run), D-022 (result). GPU run 2: D-035 (pre-registered, with a seed-confirmation rule for any pass). Refinement: D-033 (targets, protocol), D-034 (amendment and smoke-run disclosure), D-036 – D-041 (rounds 1–6 and the final verdict), D-042 (shipped implementation; post-hoc implementation check, not a selection step). Short-text experiments (exploratory): D-043, D-044, including a prediction for GPU run 2 written before its result. Six evaluation rounds on fresh data were spent; the full sequence is reported, not the best round.

Appendix B. Datasets and licenses

Data	Use	License
FLORES-200 (NLLB Team, 2022)	fitting (dev), gate and adapter tests (devtest)	CC-BY-SA-4.0
Wikipedia 2023-11-01 sample	fitting	CC-BY-SA-4.0
Tatoeba	fitting (translation scatter)	CC-BY-2.0 FR
MASSIVE (FitzGerald et al., 2023), via MTEB	§7 conversational	CC-BY-4.0
Banking77 (Casanueva et al., 2020)	§7 customer support	CC-BY-4.0
Stack Exchange titles (Geigle et al., 2021), via MTEB	§7 technical Q&A	CC-BY-SA (Stack Exchange)
arXiv titles, via MTEB	§7 science	CC0-1.0
IndicReviews (IndicXTREME), via MTEB	§7 reviews	CC0-1.0
CodeSearchNet (Husain et al., 2019), via MTEB	§7 code	MIT
MASSIVE train / validation splits	§5.4 fitting (train), §8 confirmation (validation), language identifier (train)	CC-BY-4.0
MasakhaNEWS (Adelani et al., 2023): MTEB test + original train	§8	AFL-3.0

References

Casanueva et al. Efficient Intent Detection with Dual Sentence Encoders (Banking77), NLP4ConvAI 2020 · Chen et al. BGE M3, 2024 · Doddapaneni et al. Towards Leaving No Indic Language Behind (IndicXTREME), ACL

2023 · Duquenne et al. SONAR, 2023 · Feng et al. LaBSE, ACL 2022 · FitzGerald et al. MASSIVE, ACL 2023 · Geigle et al. TWEAC, 2021 · Grootendorst. BERTopic, 2022 · Huh et al. The Platonic Representation Hypothesis, ICML 2024 · Husain et al. CodeSearchNet Challenge, 2019 · Jha et al. Harnessing the Universal Geometry of Embeddings, arXiv:2505.12540, 2025 · LCM team. Large Concept Models, 2024 · Lee et al. RQ-VAE, CVPR 2022 · Muennighoff et al. MTEB, EACL 2023 · NLLB Team. FLORES-200, 2022 · Rajput et al. TIGER, NeurIPS 2023 · Singh et al. Semantic IDs for ranking, 2024 · Tay et al. DSI, NeurIPS 2022 · Wang et al. Multilingual E5, 2024 · Zeghidour et al. SoundStream, 2021 · Bhartrhari, Vakyapadiya.